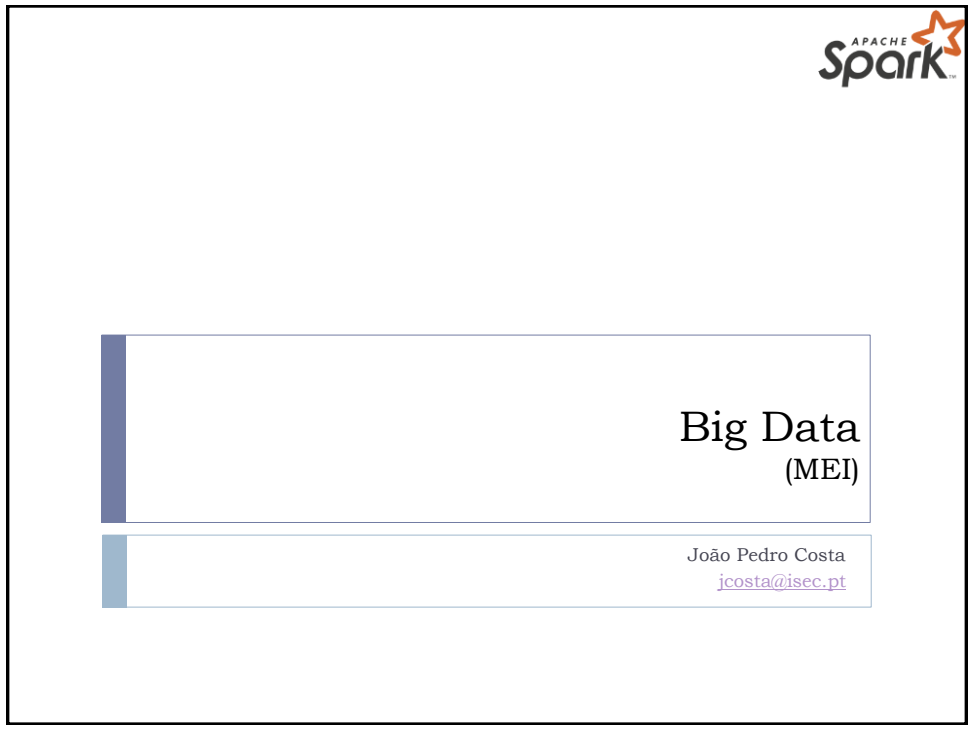


1

Batch vs Real-time processing

Batch processing

- large group of data/transaction processed in a single run
- Jobs run without manual intervention
- entire data is pre-selected and fetched using command-line parameters and scripts
- used to execute multiple operations, handle heavy data load
- offline workflow
- Ex. Regular reports for decision making

Real-time processing

- data processing starts upon data or command
- it must execute on response time within time constraints
- Ex. Fraud detection

3

Limitations of MapReduce in Hadoop

- Unsuitable for real-time processing
  - batch oriented, takes minutes, depending upon the amount of data and number of nodes
- Unsuitable for trivial operations
  - like filter and joins
- Unfit for large data on network
  - works on the data locality principle
  - requires shuffling data over the network

4

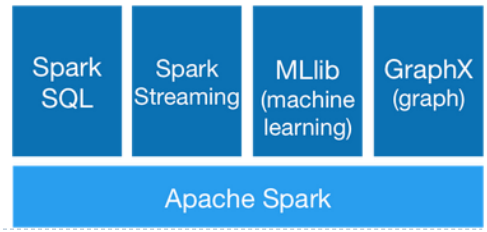
Limitations of MapReduce in Hadoop

- Unsuitable with OLTP
  - batch oriented
  - requires large number of short transactions
- Unfit for interactive execution
  - state-less execution

5

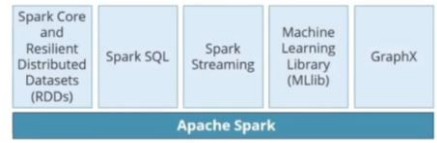
Spark

- is a unified analytics engine for large-scale data processing
- provides an optimized engine that supports general execution graphs
- provides high-level APIs in Java, Scala, Python and R
- supports a rich set of higher-level tools
  - Spark SQL for SQL and structured data processing
  - Structured Streaming for incremental computation and stream processing
  - MLlib for machine learning
  - GraphX for graph processing



Spark

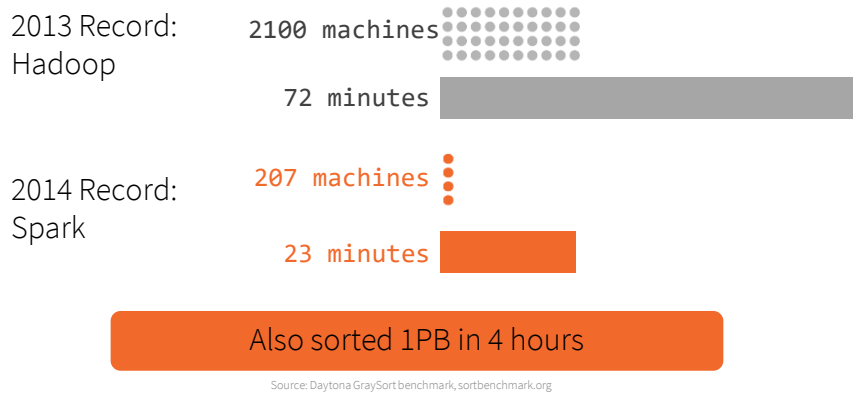
- open-source cluster computing framework
- Suitable for real-time processing,
- Trivial and processing larger data on network
- Provides up to 100x times faster performance
  - Applications with in-memory primitives
  - Compared to two-stage of map-reduce
- Suitable for machine learning algorithms
  - Allows programs to load and query data



Spark vs Hadoop

On-Disk Sort Record:

Time to sort 100TB



Also sorted 1PB in 4 hours

Source: Daytona GraySort benchmark, sortbenchmark.org

8

Spark vs Hadoop

```
time yarn jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar
pi 10 100000
 Number of Maps  = 10
 Samples per Map = 100000
 22/06/01 13:59:03 INFO mapreduce.Job:  map   0% reduce   0%
 22/06/01 13:59:11 INFO mapreduce.Job:  map  20% reduce   0%
 22/06/01 13:59:30 INFO mapreduce.Job:  map  20% reduce   7%
 22/06/01 13:59:34 INFO mapreduce.Job:  map 100% reduce   7%
 22/06/01 13:59:36 INFO mapreduce.Job:  map 100% reduce 100%

 Estimated value of Pi is 3.141552
```

```
real    0m58.159s
user    0m13.226s
sys     0m01.227s
```

```
time run-example  --master spark://10.84.128.47:7077 SparkPi 10

 Executor updated: app-20220601135521-0017/2 is now RUNNING
 Executor updated: app-20220601135521-0017/3 is now RUNNING
 Executor updated: app-20220601135521-0017/0 is now RUNNING
 Executor updated: app-20220601135521-0017/1 is now RUNNING

 Pi is roughly 3.143031
```

```
real    0m18.526s
user    0m14.225s
sys     0m00.827s
```


9

Spark vs Hadoop

```
time yarn jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar
pi 100 100000
 Number of Maps  = 100
 Samples per Map = 100000
 22/06/01 13:59:03 INFO mapreduce.Job:  map   0% reduce   0%
 22/06/01 13:59:11 INFO mapreduce.Job:  map  20% reduce   0%
 22/06/01 13:59:30 INFO mapreduce.Job:  map  20% reduce   7%
 22/06/01 13:59:34 INFO mapreduce.Job:  map 100% reduce   7%
 22/06/01 13:59:36 INFO mapreduce.Job:  map 100% reduce 100%

 Estimated value of Pi is 3.141584400
```

```
real    3m12.815s
user    0m11.842s
sys     0m01.452s
```

```
time run-example  --master spark://10.84.128.47:7077 SparkPi 100

 Executor updated: app-20220601135521-0017/2 is now RUNNING
 Executor updated: app-20220601135521-0017/3 is now RUNNING
 Executor updated: app-20220601135521-0017/0 is now RUNNING
 Executor updated: app-20220601135521-0017/1 is now RUNNING

 Pi is roughly 3.1411391141139116
```

```
real    0m19.739s
user    0m18.330s
sys     0m01.118s
```


10

Spark vs Hadoop

```
time yarn jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar
pi 1000 100000
 Number of Maps  = 100
 Samples per Map = 100000
 22/06/01 13:59:03 INFO mapreduce.Job:  map   0% reduce   0%
 22/06/01 13:59:11 INFO mapreduce.Job:  map  20% reduce   0%
 22/06/01 13:59:30 INFO mapreduce.Job:  map  20% reduce   7%
 22/06/01 13:59:34 INFO mapreduce.Job:  map 100% reduce   7%
 22/06/01 13:59:36 INFO mapreduce.Job:  map 100% reduce 100%
 Job Finished in 1571.359 seconds

 Estimated value of Pi is 3.141584400
```

```
real   26m42.591s
user    0m28.138s
sys     0m04.926s
```

```
time run-example  --master spark://10.84.128.47:7077 SparkPi 1000

 Executor updated: app-20220601135521-0017/2 is now RUNNING
 Executor updated: app-20220601135521-0017/3 is now RUNNING
 Executor updated: app-20220601135521-0017/0 is now RUNNING
 Executor updated: app-20220601135521-0017/1 is now RUNNING

 Pi is roughly 3.1415488314154882
```

```
real    0m26.481s
user    0m27.823s
sys     0m01.838s
```


11

Spark vs Hadoop

```
time yarn jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar
pi 1000 100000
  Number of Maps  = 100
  Samples per Map = 100000
  22/06/01 13:59:03 INFO mapreduce.Job:  map   0% reduce   0%
  22/06/01 13:59:11 INFO mapreduce.Job:  map  20% reduce   0%
  22/06/01 13:59:30 INFO mapreduce.Job:  map  20% reduce   7%
  22/06/01 13:59:34 INFO mapreduce.Job:  map 100% reduce   7%
  22/06/01 13:59:36 INFO mapreduce.Job:  map 100% reduce 100%
  Job Finished in 1571.359 seconds

  Estimated value of Pi is 3.141584400
```

```
real    12m53.368s
user     0m25.500s
sys      0m04.756s
```

```
time run-example  --master spark://10.84.128.47:7077 SparkPi 1000

  Executor updated: app-20220601135521-0017/2 is now RUNNING
  Executor updated: app-20220601135521-0017/3 is now RUNNING
  Executor updated: app-20220601135521-0017/0 is now RUNNING
  Executor updated: app-20220601135521-0017/1 is now RUNNING

  Pi is roughly 3.1415488314154882
```

```
real    0m26.481s
user    0m27.823s
sys     0m01.838s
```


Spark vs Hadoop

```
time yarn jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar
teragen 1000000 /datasets/terasort/input
  Number of Maps  = 2
  Samples per Map = 100000
  22/06/01 13:59:03 INFO mapreduce.Job:  map   0% reduce   0%
  22/06/01 13:59:11 INFO mapreduce.Job:  map  20% reduce   0%
  22/06/01 13:59:30 INFO mapreduce.Job:  map  20% reduce   7%
  22/06/01 13:59:34 INFO mapreduce.Job:  map 100% reduce   7%
  22/06/01 13:59:36 INFO mapreduce.Job:  map 100% reduce 100%

  Estimated value of Pi is 3.141552
```

```
real    0m28.528s
user    0m08.683s
sys     0m00.799s
```

```
time run-example  --master spark://10.84.128.47:7077 SparkPi 10

  Executor updated: app-20220601135521-0017/2 is now RUNNING
  Executor updated: app-20220601135521-0017/3 is now RUNNING
  Executor updated: app-20220601135521-0017/0 is now RUNNING
  Executor updated: app-20220601135521-0017/1 is now RUNNING

  Pi is roughly 3.143031
```

```
real    0m18.526s
user    0m14.225s
sys     0m0.0827s
```

Spark vs Hadoop

```
time yarn jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar
terasort /out/terasort/hadoop
 Number of Maps  = 10
 Samples per Map = 100000
 22/06/01 13:59:03 INFO mapreduce.Job:  map   0% reduce   0%
 22/06/01 13:59:11 INFO mapreduce.Job:  map  20% reduce   0%
 22/06/01 13:59:30 INFO mapreduce.Job:  map  20% reduce   7%
 22/06/01 13:59:34 INFO mapreduce.Job:  map 100% reduce   7%
 22/06/01 13:59:36 INFO mapreduce.Job:  map 100% reduce 100%

 Estimated value of Pi is 3.141552
```

```
real    0m39.226s
user    0m09.471s
sys     0m00.995s
```

```
time run-example  --master spark://10.84.128.47:7077 SparkPi 10

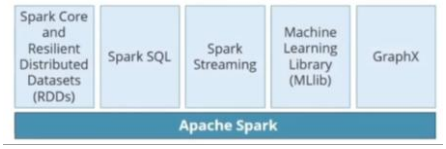
 Executor updated: app-20220601135521-0017/2 is now RUNNING
 Executor updated: app-20220601135521-0017/3 is now RUNNING
 Executor updated: app-20220601135521-0017/0 is now RUNNING
 Executor updated: app-20220601135521-0017/1 is now RUNNING

 Pi is roughly 3.143031
```

```
real    0m18.526s
user    0m14.225s
sys     0m0.0827s
```


Spark

- Open-source cluster computing framework
- Suitable for real-time processing,
- Trivial and processing larger data on network
- Provides up to 100x times faster performance
  - Applications with in-memory primitives
  - Compared to two-stage of map-reduce
- Suitable for machine learning algorithms
  - Allows programs to load and query data



In Memory processing

- In-memory **processing** (and **storage**) engine
  - Load data from HDFS, Cassandra, HBase
  - Resource management via: YARN, Mesos, Spark, Amazon EC2
- It can use Hadoop but also works standalone!
- Task scheduling and monitoring

16

In Memory processing

- Rich APIs
  - APIs for Java, Scala, Python, R
  - Thrift JDBC/ODBC server for SQL
  - High-level domain-specific tools/languages
    - Advanced APIs simplify typical computation tasks
- Interactive shells with tight integration
  - pyspark: Python
  - spark-shell: Scala (object-oriented functional language running on JVM)
  - sparkR: R (basic support)
  - spark-sql

17

RDD -Resilient Distributed Datasets

- a fundamental data structure of Apache Spark
- a fault-tolerant collection of elements that can be operated on in parallel
- are an immutable collection of objects which computes on the different nodes of the cluster
- each and every dataset in **Spark RDD** is logically partitioned across many servers so that they can be computed on different nodes of the cluster

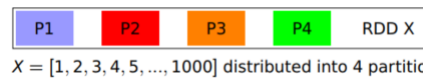
- **Resilient**
  - fault-tolerant with the help of RDD lineage graph (DAG)
  - able to recompute missing or damaged partitions due to node failures
- **Distributed**
  - data resides on multiple nodes
- **Dataset** represents records of the data you work with
  - data can be loaded externally

18

RDD -Resilient Distributed Datasets

- named collection of elements distributed in partitions
- typically a list or a map (key-value pairs)
- a RDD is immutable
- high-level APIs provide additional representations
- shared variables offer shared memory access
- durability of data
  - RDDs live until the SparkContext is terminated
  - to keep them, they need to be persisted (e.g., to HDFS)

- fault-tolerance is achieved by **re-computing** the data



$X = [1, 2, 3, 4, 5, \ldots, 1000]$ distributed into 4 partitions

19

RDD -Resilient Distributed Datasets

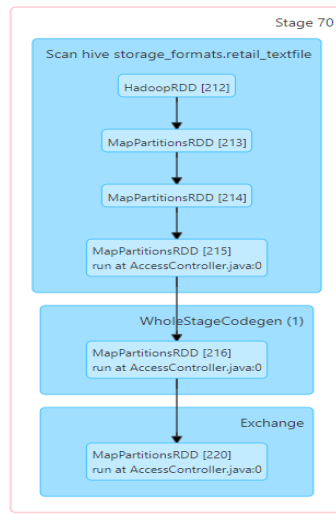
- RDDs can be transformed into new derived (newly named) RDDs

```
rdd2 = rdd.filter(lambda x : (x % 2 == 0) ) # filter odd tuples
```

- computation runs in parallel on the partitions
- usually re-computed, but RDD can be kept in memory or stored if large
- RDDs can be redistributed (called shuffle)
- knows its data lineage (how it was computed)
- fault-tolerant collection of elements (lists, dictionaries)
  - split into number of partitions and distributed
  - derived RDDs can be re-computed by using the recorded lineage

20

RDD -Resilient Distributed Datasets



21

RDD -Features

In-memory Computation

- stores intermediate results in distributed memory instead of stable storage

Lazy Evaluations

- all transformations are lazy, results are not computed right away
- Instead, they remember the transformations applied to some base data set
- transformations are computed when an action requires a result for the driver

Fault Tolerance

- Spark RDDs are fault tolerant as they track data lineage information to rebuild lost data automatically on failure.
- lost data is rebuild on failure using lineage
  - each RDD remembers how it was created from other datasets (by transformations like a map, join or groupBy) to recreate itself

RDD -Features

Immutability

- data is safe to share across processes
- can be created or retrieved anytime which makes caching, sharing & replication easy
- it is a way to reach consistency in computations.

Partitioning

- fundamental unit of parallelism in Spark RDD
- each partition is one logical division of data which is mutable
- can create a partition through some transformations on existing partitions

RDD -Features

Persistence

- users can state which RDDs they will reuse and choose a storage strategy for them (e.g., in-memory storage or on Disk)

Coarse-grained Operations

- applies to all elements in datasets through maps, filter or group by operations

Location-Stickiness

- RDDs are capable of defining placement preference to compute partitions.
- placement preference refers to information about the location of RDD
- **DAGScheduler** places partitions so that a task is close to data
  - to speed up computation

Spark - Pi Example

```
import sys
from random import random
from operator import add
from pyspark.sql import SparkSession

spark = SparkSession.builder\
        .appName("PythonPi")\
        .getOrCreate()
partitions = 10 # partitions = int(sys.argv[1]) if len(sys.argv) > 1 else 2
n = 100000 * partitions

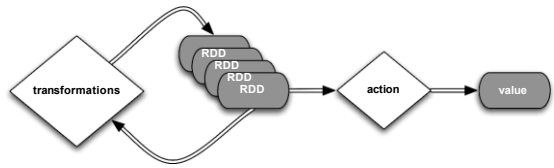
def f(_):
   x = random() * 2 - 1
   y = random() * 2 - 1
   return 1 if x ** 2 + y ** 2 <= 1 else 0
#count=spark.sparkContext.parallelize(range(1, n+1), partitions).map(f).reduce(add,
baseRDD = spark.sparkContext.parallelize(range(1, n + 1), partitions)   # base RDD
tRDD = baseRDD.map(f)                                                   # transformed RDD
count = tRDD.reduce(add)                                                #   action
print("Pi is roughly %f" % (4.0 * count / n))
spark.stop()
```

```
Pi is roughly 3.146360
```

Spark - Pi Example

```
#count=spark.sparkContext.parallelize(range(1, n+1), partitions).map(f).reduce(add)
baseRDD = spark.sparkContext.parallelize(range(1, n + 1), partitions)   # base RDD
tRDD = baseRDD.map(f)                                                   # transformed RDD
count = tRDD.reduce(add)                                                #    action
print("Pi is roughly %f" % (4.0 * count / n))
spark.stop()
```

```
Pi is roughly 3.146360
```



Spark – WordCount Example

```
from pyspark.sql import SparkSession
#from pyspark import SparkConf
spark = SparkSession.builder\
        .appName('Spark WordCount')\
        .getOrCreate()
```

```
sc = spark.sparkContext
sc
```

```
lines =  sc.textFile("/datasets/books/shakespeare.txt")
lines
```

```
words = lines.flatMap(lambda s: s.split(" "))
words
```

```
pairs = words.map(lambda s: (s,1))
pairs
```

```
counts = pairs.reduceByKey(lambda a, b: a + b)
counts
```

```
counts.take(5)
```

```
#spark.stop()
```

Spark – WordCount Example

```
from pyspark.sql import SparkSession
spark = SparkSession.builder\
        .appName('Spark WordCount')\
        .getOrCreate()
```

```
sc = spark.sparkContext
sc
```

```
<div> ●●●
```

```
lines =  sc.textFile("/datasets/books/shakespeare.txt")
lines
```

```
/datasets/books/shakespeare.txt MapPartitionsRDD[1] at textFile at NativeMethodAccessorImpl.java:0
```

```
words = lines.flatMap(lambda s: s.split(" "))
words
```

```
PythonRDD[2] at RDD at PythonRDD.scala:53
```

```
pairs = words.map(lambda s: (s,1))
pairs
```

```
PythonRDD[3] at RDD at PythonRDD.scala:53
```

```
counts = pairs.reduceByKey(lambda a, b: a + b)
counts
```

```
PythonRDD[8] at RDD at PythonRDD.scala:53
```

```
counts.take(5)
```

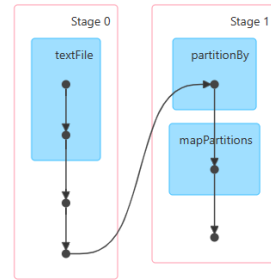
```
[('The', 4630),
 ('Project', 79),
 ('of', 17111),
 ('Shakespeare', 5),
 ('', 62184)]
```

```
spark.stop()
```

Details for Job 0

Status: SUCCEEDED
Submitted: 2024/05/02 15:57:08
Duration: 4 s
Completed Stages: 2

▸ Event Timeline
▾ DAG Visualization



RDD

```
# only one SparkContext at a time
import pyspark
sc = pyspark.SparkContext('local[*]')
sc.setLogLevel("ERROR")
```

```
lines = sc.textFile('/datasets/books/shakespeare.txt')
lines
```

```
lineLength = lines.map(lambda line: len(line))
lineLength
```

```
totalLength = lineLength.reduce(lambda a,b: a +b)
totalLength
```

```
sc.stop()
```

- What is the result?
- When is it executed?

RDD

```
# only one SparkContext at a time
import pyspark
sc = pyspark.SparkContext('local[*]')
sc.setLogLevel("ERROR")
```

```
lines = sc.textFile('/datasets/books/shakespeare.txt')
lines
```

```
/datasets/books/shakespeare.txt MapPartitionsRDD[1] at textFile at NativeM
```

```
lineLength = lines.map(lambda line: len(line))
lineLength
```

```
PythonRDD[2] at RDD at PythonRDD.scala:53
```

```
totalLength = lineLength.reduce(lambda a,b: a +b)
totalLength
```

```
5333743
```

```
sc.stop()
```

- What is the result?
- When is it executed?

RDD

Lambda expressions

- simple functions that can be written as an expression
- do not support multi-statement functions or statements that do not return a value

driver program must pass functions to run on the cluster, using:

- Lambda expressions
- Local defs inside the function calling into Spark, for longer code.
- Top-level functions in a module.

RDD

WordCount

- Local defs inside the function calling into Spark, for longer code.

```
def myFunc(s):
   words = s.split(" ")
   return len(words)
```

```
def mySum(a,b):
   return a + b
```

```
nWords = sc.textFile('/datasets/books/shakespeare.txt') \
           .map(myFunc) \
           .reduce(mySum)
print(nWords)
```

```
[Stage 9:>
1418390
```

- Lambda expression

```
nWords = sc.textFile('/datasets/books/shakespeare.txt') \
           .map(lambda line : len(line.split(" "))) \
           .reduce(lambda a,b: a+ b)
print(nWords)
```

```
1418390
```


RDD

Parallelize Collections

- The elements of the collection are copied to form a distributed dataset that can be operated on in parallel.
- created by calling SparkContext's parallelize method on an existing iterable or collection in your driver program
- once created, the distributed dataset can be operated on in parallel
- Typically, Spark will run one task for each partition of the cluster.

```
sc.parallelize( collection , num_partitions)
```

```
data = [1, 2, 3, 4, 5]
distData = sc.parallelize(data)

distData.reduce(lambda a, b: a + b)
```

RDD - Transformations

filter(*func*)	*Return a new dataset with source elements on which func returns true*
map(*func*)	*Return a new distributed dataset formed by passing each element of the source through a function func*
flatMap(*func*)	*Similar to map, but each input item can be mapped to 0 or more output items (so func should return a Seq rather than a single item).*
mapPartitions(*func*)	*Similar to map, but runs separately on each partition (block) of the RDD, so func must be of type Iterator<T> => Iterator<U> when running on an RDD of type T*
mapPartitionsWithIndex(*func*)	*Similar to mapPartitions, but also provides func with an integer value representing the index of the partition, so func must be of type (Int, Iterator<T>) => Iterator<U> when running on an RDD of type T*

35

RDD – flat and flatMap

```
from pyspark import SparkContext
sc = SparkContext( appName='app1',
        master='spark://10.84.128.47:7077')
```

```
Setting default log level to "WARN". ●●●
```

```
text =['first phrase','second phrase','last']
text
```

```
['first phrase', 'second phrase', 'last']
```

```
rdd = sc.parallelize(text)
rdd
```

```
ParallelCollectionRDD[0] at readRDDFromFile at PythonRDD.scala:289
```

```
m = rdd.map(lambda l: l.split())
m.collect()
```

```
[['first', 'phrase'], ['second', 'phrase'], ['last']]
```

```
fm = rdd.flatMap(lambda l: l.split())
fm.collect()
```

```
['first', 'phrase', 'second', 'phrase', 'last']
```

```
splitWords = [ l.split() for l in text]
splitWords
```

```
[['first', 'phrase'], ['second', 'phrase'], ['last']]
```


36

RDD- Transformations

union(*otherDataset*)	*Return a new dataset containing the union of the elements in source dataset and argument*
intersection(*otherDataset*)	*Return a new RDD that contains the intersection of elements in source dataset and argument*
cartesian(*otherDataset*)	*on datasets T and U, returns a dataset of (T, U) pairs (all pairs of elements)*
join(*otherDataset,[numPartitions]*)	*on datasets of type (K, V) and (K, W), returns a dataset of (K, (V, W)) pairs with all pairs of elements for each key. Outer joins are supported through leftOuterJoin, rightOuterJoin, and fullOuterJoin.*
distinct(*[numPartitions]*))	*Return a new dataset that contains the distinct elements of the source dataset.*

```
x = sc.parallelize([("a", 1), ("b", 4)])
y = sc.parallelize([("a", 2), ("a", 3)])
sorted(x.join(y).collect())

[('a', (1, 2)), ('a', (1, 3))]
```


37

RDD- Transformations

sample(*withReplacement, fraction, seed*)	*Sample a fraction fraction of the data, with or without replacement, using a given random number generator seed*
coalesce(*numPartitions*)	*Decrease the number of partitions in the RDD to numPartitions. Useful for running operations more efficiently after filtering down a large dataset*
repartition(*numPartitions*)	*Reshuffle the data in the RDD randomly to create either more or fewer partitions and balance it across them. This always shuffles all data over the network.*

38

RDD- Actions

collect()	Return all the elements of the dataset as an array at the driver program. Useful after a filter or other operation that returns a sufficiently small subset of the data.
count()	Return the number of elements in the dataset.
first()	Return the first element of the dataset (similar to take(1))
take(*n*)	Return an array with the first *n* elements of the dataset.
takeSample(*withReplacement, num, [seed]*)	Return an array with a random sample of *num* elements of the dataset, with or without replacement, optionally pre-specifying a random number generator seed.
takeOrdered(*n, [ordering]*)	Return the first *n* elements of the RDD using either their natural order or a custom comparator.

41

RDD- Actions

countByKey()	Only available on RDDs of type (K, V). Returns a hashmap of (K, Int) pairs with the count of each key.
reduce(*func*)	Aggregate the elements of the dataset using a function *func* (which takes two arguments and returns one). The function should be commutative and associative so that it can be computed correctly in parallel.
foreach(*func*)	Run a function *func* on each element of the dataset. This is usually done for side effects such as updating an Accumulator or interacting with external storage systems. Note: modifying variables other than Accumulators outside of the foreach() may result in undefined behavior.
saveAsTextFile(*path*)	Write the elements of the dataset as a text file (or set of text files) in a given directory in the local filesystem, HDFS or any other Hadoop-supported file system. Spark will call toString on each element to convert it to a line of text in the file.

42

PySpark

- Pi Calculus

```
import pyspark
import random
conf = pyspark.SparkConf() \
    .setAppName('App pi') \
    .setMaster("local[2]")\
    .set("spark.ui.enabled","false")

sc = pyspark.SparkContext(conf=conf)

NUM_SAMPLES=1000
def inside(p):
    x, y = random.random(), random.random()
    return x*x + y*y < 1

count = sc.parallelize(range(0, NUM_SAMPLES)) \
             .filter(inside).count()
print("Pi is roughly  %f" % (4.0 * count / NUM_SAMPLES))
sc.stop()
```

```
Pi is roughly  3.196000
```


RDD- examples

```
# only one SparkContext at a time
from pyspark.sql import SparkSession
spark = SparkSession.builder.master("local[1]").getOrCreate()
sc = spark.sparkContext
sc.setLogLevel("ERROR")
```

```
rdd = sc.textFile('/datasets/books/shakespeare.txt')
rdd.foreach(print) ## each line
```

```
This is the 100th Etext file presented by Project Gutenberg, and (0 + 1) / 1] ●●●
```

```
#Flatmap
rdd2=rdd.flatMap(lambda x: x.split(" "))
rdd2.foreach(print) ## each word
```

```
This ●●●
```

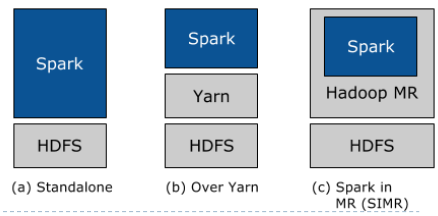
RDD- examples

- What is the result?

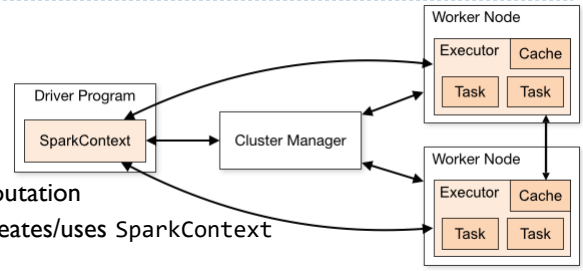
```
txt = sc.textFile('/datasets/books/shakespeare.txt')
wordCount = txt.flatMap(lambda line: line.lower().split(" "))\
  .map(lambda word: (word, 1))\
  .reduceByKey(lambda a,b:a +b) \
  .filter(lambda a : len(a[0])>= 5) \
  .sortBy(lambda a:a[1], ascending=False)\
  .collect()
print(wordCount[:10])
```


Spark vs Hadoop

- Spark is intended to *enhance*, not replace, the Hadoop stack
- Spark designed to read and write data from and to HDFS
- Hadoop users can combine Spark with MapReduce, HBase, …
- Spark can be deployed in different ways in a Hadoop cluster:
  - Standalone
  - YARN
  - SIMR (Spark in MapReduce) (Depreacted)
  - Kubernetes



Spark - Architecture



Driver program

- controls data flow/computation
- process runs `main()`, creates/uses `SparkContext`

Executor process

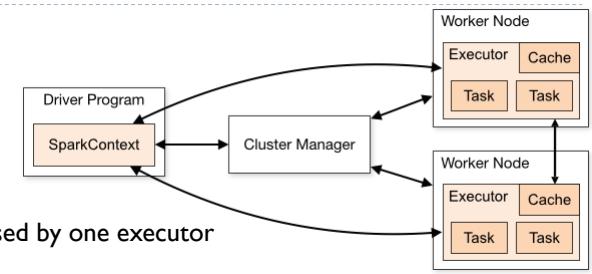
- are spawned on nodes
- provides slots to runs tasks
- store and manage RDDs
- performs computation (usually on local partition)
- isolates apps, thus data cannot be shared between apps

Cluster manager

- allocates cluster resources and runs executor

48

Spark - Architecture



Task

- a unit of work processed by one executor

Stage

- collection of tasks executing the same code
- run concurrently
- works independently on partitions without data shuffling

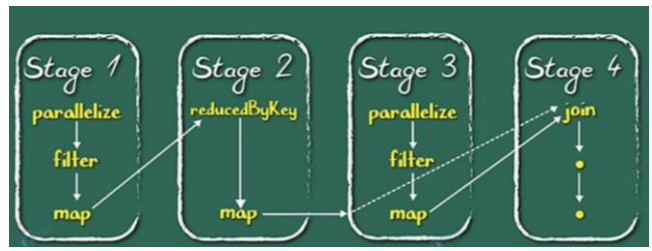
Job

- a spark action triggering a computation starts a job

49

Spark - Architecture

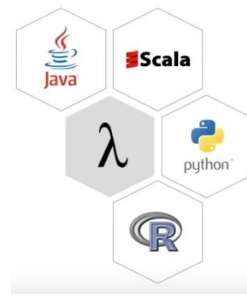
`Job` **Expressed as Directed Acyclic Graph (DAG)**



Language Flexibility

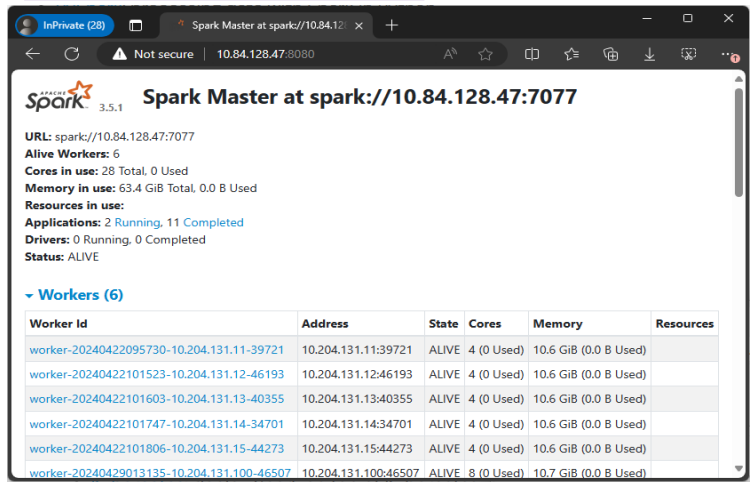
- MapReduce → mainly Java
- Spark supports
  - Java, Scala and Python

  Able to define lambda functions



Spark UI

- http://10.84.128.47:8080/



Spark UI

- http://10.84.128.47:4040/

